

Objective: Deploy and respond to a variety of different cyberattack vectors in a simulated environment

Machines:

Lab network:

Ubuntu 22.04 home lab

Desktop-SM

Defense-client-1

Wazuh host vps

Kali VM on desktop SM

Tools:

Wazuh

Ubuntu/Kali

Hydra

Ssh

Most of the machines on this network are ones I already own: the home lab is a 24/7 uptime minipc that serves as a network file share along with various game server hosting, desktop-SM is my personal, and the host VPS is solely for wazuh-dashboard and wazuh-manager. I've spun up a ubuntu 24.04 instance on a digitalocean VM using their default image to use as my target machine for this lab / simulation.

Defense-client-1: an ubuntu 24.04 server with 1gb ram, 1vcpu, and 25gb ssd on a digitalocean droplet

–Spun up a default ubuntu 24.04 server to use as a defense-client-1 and installed wazuh agent and connected it to my manager

-Did initial vulnerability checks and repair based on agent analysis: first did kernel upgrade to mitigate wazuh detected CVE-2026-43304, CVE-2026-43037, among others related to an outdated kernel from the default digital ocean 24.04 image. Updated running kernel to 6.8.0-124 (latest as of 6/29/26)

–initial state of VM defense client: all ports open, password based ssh authentication

Vulnerability details

# package.size	15,033,344
t package.type	deb
t package.version	6.8.0-71.71
t vulnerability.category	Packages
t vulnerability.classification	CVSS
t vulnerability.description	In the Linux kernel, the following vulnerability has been resolved:libceph: define and enforce CEPH_MAX_KEY_LEN when decoding the key, verify that the key material would fit into a fixed-size buffer in process_auth_done() and generally has a sane length. The new CEPH_MAX_KEY_LEN check replaces the existing check for a key with no key material which is a) not universal since CEPH_CRYPTO_MD5 has to be excluded and b) doesn't provide much value since a smaller than needed key is just as invalid as no key -- this has to be handled elsewhere anyway.
vulnerability.detected_at	Jun 29, 2026 @ 15:21:43.616
t vulnerability.enumeration	CVE
t vulnerability.id	CVE-2026-43384
vulnerability.published_at	May 8, 2026 @ 07:16:37.000
t vulnerability.reference	https://ubuntu.com/security/CVE-2026-43384 , https://www.cve.org/CVERecord?id=CVE-2026-43384
t vulnerability.scanner.condition	Package default status
t vulnerability.scanner.reference	https://cti.wazuh.com/vulnerabilities/cves/CVE-2026-43384
t vulnerability.scanner.source	Canonical Security Tracker
t vulnerability.scanner.vendor	Wazuh
# vulnerability.score.base	9.8
t vulnerability.score.version	3.1
t vulnerability.severity	Critical
vulnerability.under_evaluation	False

Start of attack procedure:

From kali VM:

Open nmap, initiate intense scan on DC1 (defense client 1)

—found open port 22 on TCP - ubuntu identifier

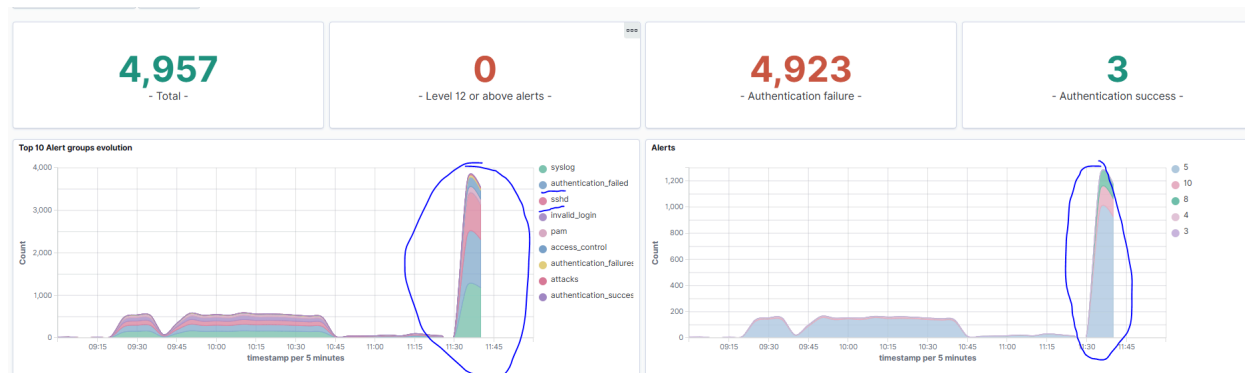
—initialized hydra and began brute force attack

—used usernames (root, admin, ubuntu) for most common for nmap identification of OS

—password list: NCSC 100k most used passwords

From defense perspective:

Monitoring defense-client-1, noticed huge inbound spike in failed authentications through threat hunting events manager



Noted majority are failed authentication / sshd related, syslog as presumed derivative from attack

Also noted a large spike in events count evolution off of the dashboard, and a large ATT&CK Credential Access spike

MITRE ATT&CK



Top Tactics

Credential Access

26662

Lateral Movement

17310

Defense Evasion

10

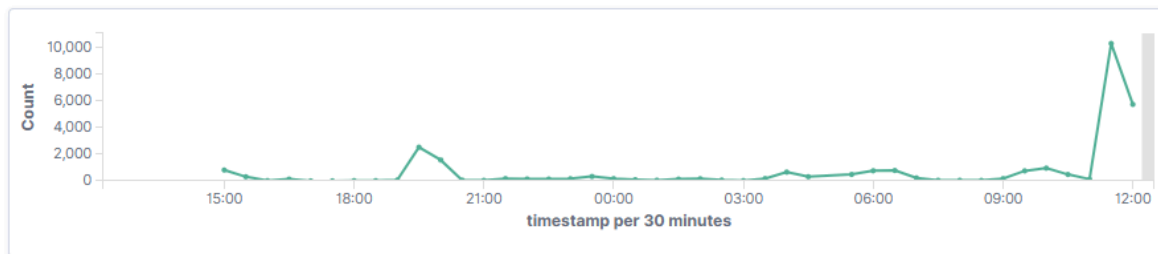
Privilege Escalation

9

Initial Access

6

Events count evolution



Determined a possible brute force attack through rule.id: "5710" OR rule.id: "5712" (5710 is auth failed, while 5712 is brute force indicator: both are logical things to check in the event of a massive spike in auth failed events)

Moved to reading individual event logs to gather data: determined the vast majority of authentication requests were originating from (162.226.4.35) (my home IP) while cycling through source ports for the outgoing connection

Table JSON

†	GeoLocation.city_name	Lakewood
†	GeoLocation.country_name	United States
🌐	GeoLocation.location	{ "lon": -81.8037, "lat": 41.4866 }
†	GeoLocation.region_name	Ohio
†	_index	wazuh-alerts-4.x-2026.06.30
†	agent.id	003
†	agent.ip	167.172.144.147
†	agent.name	defense-client-1
†	data.dstuser	root
†	data.srcip	162.226.4.35
†	data.srcport	58892
†	decoder.name	sshd
†	decoder.parent	sshd

Table JSON

f	GeoLocation.city_name	Lakewood
f	GeoLocation.country_name	United States
⊕	GeoLocation.location	{ "lon": -81.8037, "lat": 41.4866 }
f	GeoLocation.region_name	Ohio
f	_index	wazuh-alerts-4.x-2026.06.30
f	agent.id	003
f	agent.ip	167.172.144.147
f	agent.name	defense-client-1
f	data.dstuser	root
f	data.srcip	162.226.4.35
f	data.srcport	58904

^^these two images represent snaps of the specific sshd auth failure logs to show the repeated scr ip consistency with varying source ports, but several other types of logs also showed the source ip as the issue, such as pluggable authentication module logs indicating an access failure.

Also noted a consistent dstuser: it appears I am only trying to guess the password for the root account in this brute force attack, assuming it exists (of course I know it exists and I set up the attack vector, but this is from psuedo perspective of someone just defending a system)

Okay, now my thought process: I can see that this attack is coming in, and has thus far been unsuccessful, but I need an immediately detection and response. My determination is to design a new wazuh rule that immediately throws a high severity alert, level 12, if the brute force succeeds. I've determined it'd be best practice to implement two detection rules: one for 5720 into 5715 (multiple failed authentication attempts into a successful authentication attempt) and for 5712 into 5715 (brute force on non-existent user into successful authentication attempt - in my eyes, less likely, and more likely to be caught by the primary rule, but good as a fail safe for edge case brute force scenarios. As is standard practice, I set MITRE

```
<group name="linux,sshd,">  
  <rule id="100001" level="12">  
    <if_matched_sid>5712</if_matched_sid>
```

```

<if_sid>5715</if_sid>
<description>Brute force: non existent user into successful auth</description>
<mitre>
  <id>T1110</id>
</mitre>
</rule>
</group>

```

```

<group name="linux,sshd,">
  <rule id="100002" level="12">
    <if_matched_sid>5712</if_matched_sid>
    <if_sid>5715</if_sid>
    <description>Brute force: multiplied failed into successful auth</description>
    <mitre>
      <id>T1110</id>
    </mitre>
  </rule>
</group>

```

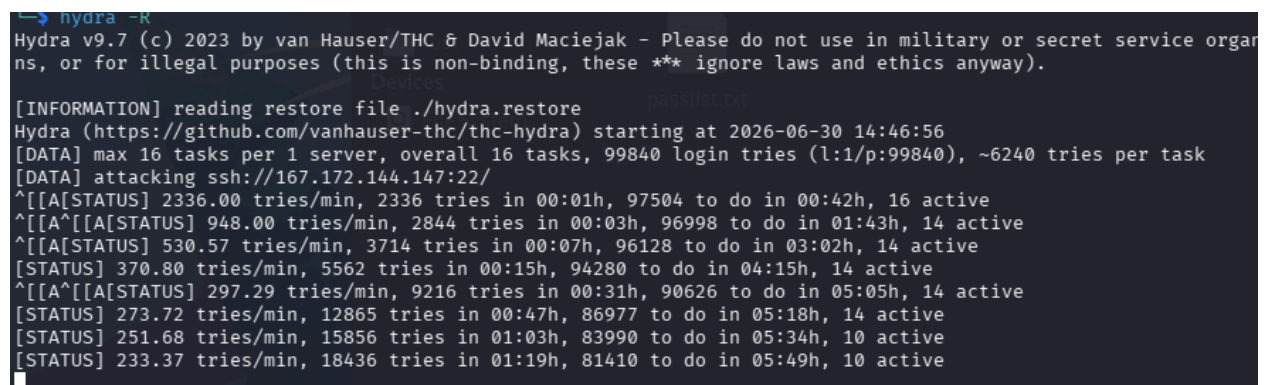
^^^the above code creates a new rule alert for each

— now in a production environment, I'd want to handle this directly, so I built in an automatic response dropping any connection to the source IP for 10 minutes in the event of a detected brute force attack, on either an existing or non existing user (signified by 5712 and 5720):

```

<active-response>
  <command>firewall-drop</command>
  <location>local</location>
  <rules_id>5712,5720</rules_id> <timeout>600</timeout>
</active-response>

```



```

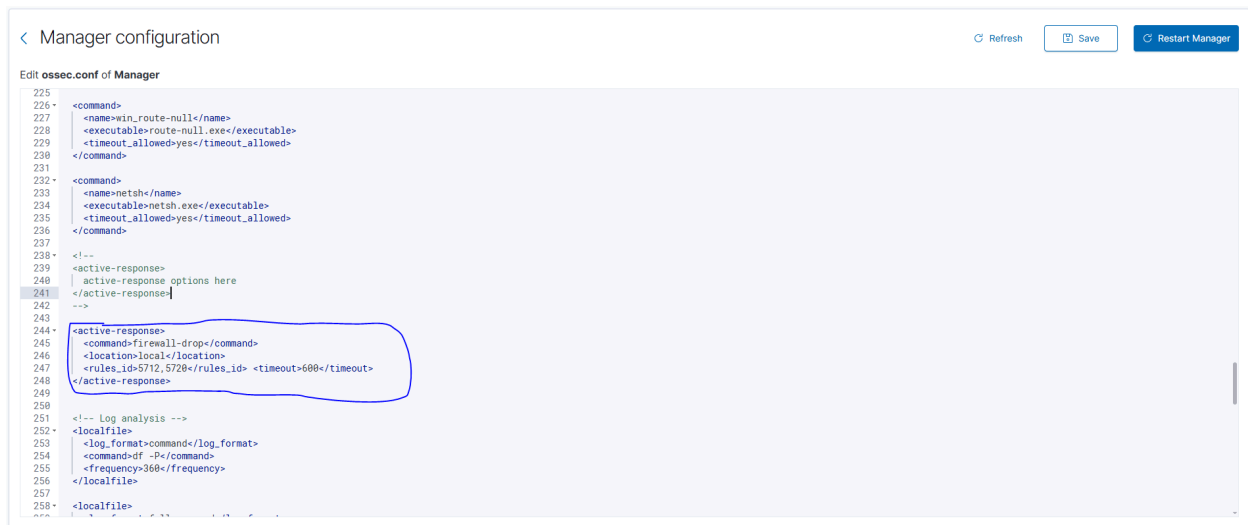
→ hydra -R
Hydra v9.7 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organs, or for illegal purposes (this is non-binding, these *** ignore laws and ethics anyway).

[INFORMATION] reading restore file ./hydra.restore
Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-06-30 14:46:56
[DATA] max 16 tasks per 1 server, overall 16 tasks, 99840 login tries (l:1/p:99840), ~6240 tries per task
[DATA] attacking ssh://167.172.144.147:22/
^[[A[STATUS] 2336.00 tries/min, 2336 tries in 00:01h, 97504 to do in 00:42h, 16 active
^[[A^[[A[STATUS] 948.00 tries/min, 2844 tries in 00:03h, 96998 to do in 01:43h, 14 active
^[[A[STATUS] 530.57 tries/min, 3714 tries in 00:07h, 96128 to do in 03:02h, 14 active
[STATUS] 370.80 tries/min, 5562 tries in 00:15h, 94280 to do in 04:15h, 14 active
^[[A^[[A[STATUS] 297.29 tries/min, 9216 tries in 00:31h, 90626 to do in 05:05h, 14 active
[STATUS] 273.72 tries/min, 12865 tries in 00:47h, 86977 to do in 05:18h, 14 active
[STATUS] 251.68 tries/min, 15856 tries in 01:03h, 83990 to do in 05:34h, 10 active
[STATUS] 233.37 tries/min, 18436 tries in 01:19h, 81410 to do in 05:49h, 10 active

```

—this is a screenshot from my hydra log from prior to instituting the active response rule, where you can see the brute force attack is proceeding as expected

Screenshot for demo purposes (specifically of adding the active response rule):



—at this point in my lab is the first time I ran into any real trouble, as even though I created the wazuh active response config I couldn't see any indication that it was actually taking effect - hydra continued to record attempts with no seeming difference, and the events manager continued to pick up sshd attempts. I identified that the wazuh agent was still online, that the wazuh manager had been restarted after adding the rule, and that firewall drop was a correctly defined command



Screenshot identifying that no events came up for rule id 5720

5712 appears to be working fine, I'm seeing lots of password spraying attacks from around the web (not bad to note, since I've started this I've noted a lot of brute force attacks from various geographical locations, last one was targeting username "server" from bulgaria).

I've noted that 40111 is firing correctly, maximum authentication attempts exceeded - not sshd unitary, but also a key indicator, so I'm trying to figure out why one is firing but not the other.

5720 should be firing as I can see the source ip and source user are correctly being read during the attack vector.

I still haven't figured out why rule 5720 isn't appearing as expected, but since I'm simulating a real world defense situation, I'm attempting to secure the network to the best of my ability before trying to fine tune my solution. I can see that the PAM rule (5551) is directly firing and should serve as a more than valid indicator to handle this situation, so I've opted to add that to my manager active response rule and restart it.

```
<active-response>
  <command>firewall-drop</command>
  <location>local</location>
  <rules_id>5712,5720,5551</rules_id> <timeout>600</timeout>
</active-response>
```

^^screenshot showing modified rule

Looks like adding 5551 correctly stopped the brute force attack on my system

```
(kali@kali)-[~]
$ hydra -R
Hydra v9.7 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organizations, or for illegal purposes (this is non-binding, these *** ignore laws and ethics anyway).

[INFORMATION] reading restore file ./hydra.restore
Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-06-30 16:55:11
[DATA] max 16 tasks per 1 server, overall 16 tasks, 99840 login tries (l:1/p:99840), ~6240 tries per task
[DATA] attacking ssh://167.172.144.147:22/
[STATUS] 27564.00 tries/min, 27564 tries in 00:01h, 72286 to do in 00:03h, 6 active
[ERROR] all children were disabled due too many connection errors
0 of 1 target completed, 0 valid password found
[INFO] Writing restore file because 2 server scans could not be completed
The session file ./hydra.restore was written. Type "hydra -R" to resume session.
[ERROR] 1 target was disabled because of too many errors
[ERROR] 1 targets did not complete
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2026-06-30 16:56:17
```

Screenshot showing resume session failed

```
Hydra v9.7 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organizations, or for illegal purposes (this is non-binding, these *** ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-06-30 16:58:12
[WARNING] Many SSH configurations limit the number of parallel tasks, it is recommended to use option -t
[WARNING] Restorefile (you have 10 seconds to abort... (use option -I to skip waiting, to prevent overwriting, ./hydra.restore
[DATA] max 16 tasks per 1 server, overall 16 tasks, 99840 login tries (l:1/p:99840)
[DATA] attacking ssh://167.172.144.147:22/
[ERROR] could not connect to ssh://167.172.144.147:22 - Connection refused
```

Screenshot showing that the wazuh active response rule I wrote correctly identified and blocked the brute force attack. I ran this again to confirm the active response had worked correctly.

	Jun 30, 2026 @ 13:55:15.1...	defense-client-1	Host Blocked by firewall-drop Active Response
	Jun 30, 2026 @ 13:55:15.1...	defense-client-1	Host Blocked by firewall-drop Active Response

Screenshot showing active response configuration in action

After locking down the network, I've established why 5720 wasn't working: for my configuration, the correct rule id is 5763, identified by reading event logs

	Jun 30, 2026 @ 13:55:16.0...	defense-client-1	sshd: brute force trying to get access to the system. Authentication failed.	10	5763
---	------------------------------	------------------	--	----	------

Lastly, I thought about what actionable steps I had taken: I had set up the rules and the active response after identifying a threat, but who's to say the threat actor hadn't previously gained access to the system via this vector before I did this?

To ensure that hadn't happened, I pulled the logs and made sure that there were no events in the past week with a rule id indicating a successful ssh connection and a srcip of the attacker I had already identified